Real-World Engineering Challenges #6: Migrations

Lessons on migration projects from Box, Stripe, Pinterest, Doordash, LinkedIn and others.

GERGELY OROSZ
OCT 11, 2022

👋 *Hi, this is Gergely with the monthly free issue of the Pragmatic Engineer Newsletter. If you're not a full subscriber yet, you missed How Uber is measuring engineering productivity, the one on Consolidating technologies, and a few other issues. Subscribe to get this newsletter every week* 👇

We covered Migrations Done Well in an earlier issue, where we dived deep into what typical migrations are, how to execute one, and the people side of migrations. Today's Real-World Engineering Challenges focuses on this often under-discussed topic, which becomes increasingly important as companies grow and need to migrate to new systems. We cover:

1. **Box: a zero downtime data migration.** The company moved their data storage from HBase to Google Cloud Bigtable, utilizing dual writing and a 6-step migration plan. What were these steps, and why were there so many of them?
2. **Pinterest: data migration using double writes.** Pinterest also migrated away from their HBase storage to another solution. A team used double writes and a 7-step migration plan that shared many similarities with Box's. Coincidence?
3. **LinkedIn: navigating the migration chaos.** What happens when a migration needs 100+ engineers to write code, and 600+ use cases need to be moved over to the new system? Chaos, for one. Following this migration, the team shared learnings on how to make large migrations less painful.

4. **Stripe: executing a migration at scale.** A column in one table needed to be migrated to its own table. Sounds simple, right? The complexity was Stripe's scale: hundreds of millions of columns, lots of dependencies on the original table, and no downtime allowed. What approach worked?
5. **Spotify: migration lessons.** After years of executing migrations, Spotify shared important lessons. Why do product managers own migrations, and what role do product marketing managers play?
6. **DoorDash: executing a migration safely.** DoorDash moved their sessions functionality from the monolith to a microservice and wanted to make sure nothing went wrong. So they used 'magic cookies', kill switches and experimentation throughout the rollout.

1. A zero downtime data migration at Box

Box decided to move its on-premise **HBase** databases to Google Cloud Bigtable (CBT). HBase is a column-oriented, non-relational database running on top of Hadoop Distributed File System (HDFS), and Cloud Bigtable is a fully managed, scalable NoSQL database, offering up to five nines (99.999%) of availability.

Axatha Jalimarada, staff software engineer at Box, wrote about what she learned from how this migration was executed.

Box decided on the move thanks to the fully managed nature of Cloud Bigtable. Because CBCloud Bigtable had an HBase compatible client, application-level changes were minimal.

The first migration project was moving **80TB** worth of HBase Data: **200 billion keys** accessed at **80,000 reads/sec** and **20,000 writes/sec** as part of the storage of the encryption keys that Box assigns each file as part of their metadata storage platform.

The company's requirements were:

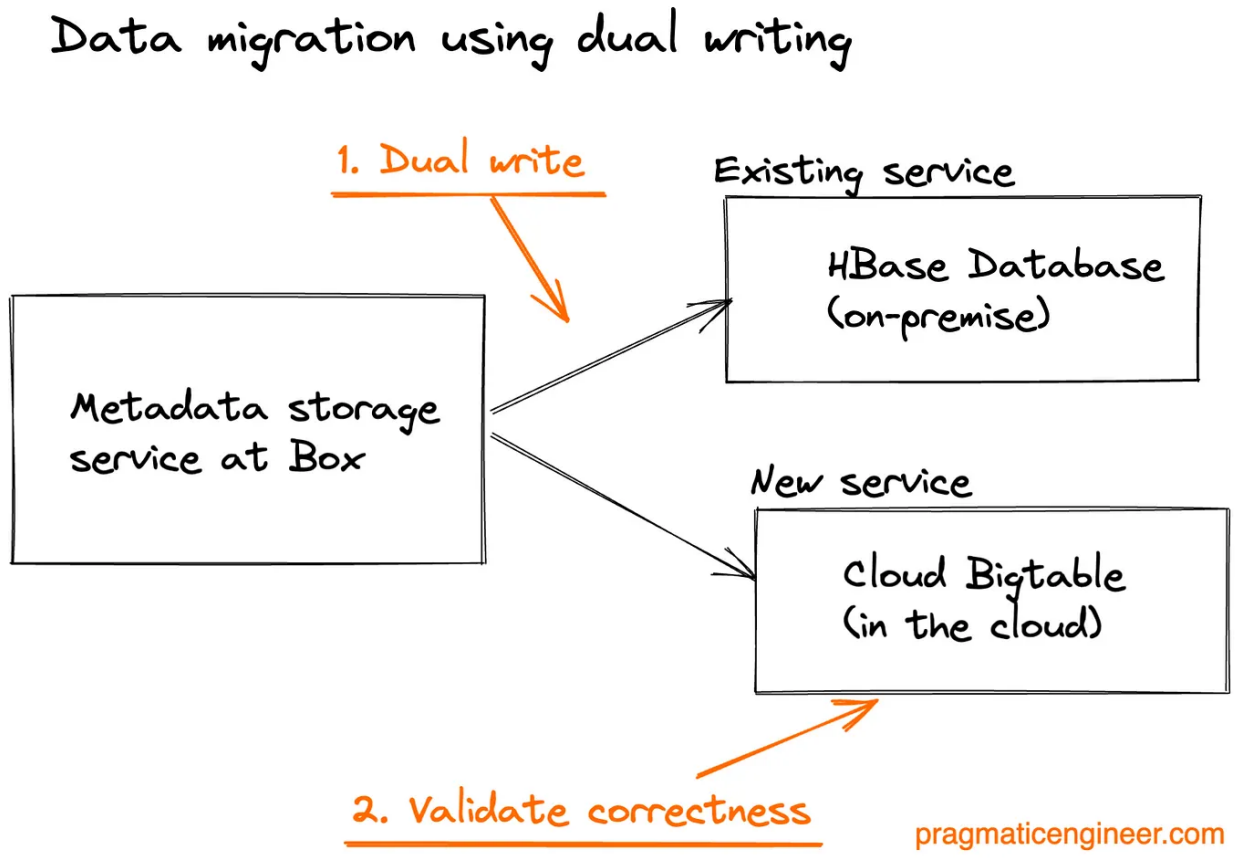
- **Zero downtime migration.** If there was downtime, customers couldn't have accessed files.
- **Latency cannot change.** The team did not want the service's latency to increase.

- **Strong guarantee of data integrity.** Any incorrect data would mean a file could not be accessed, which was not an option.

Here's how the team at Box went about the migration:

1. Validate the performance of Cloud Bigtable. Box did not rely on numbers provided by Google on the performance of Cloud Bigtable. Instead, they did their homework. And they were right to do so! When it came to using **HDDs** (hard disk drives) or **SSDs** (solid states drives), Cloud Bigtable had not just different performance characteristics, but also different cost levels. Read and write latencies were 20x lower when using SSDs, and the team made that choice.

2. Proof of concept. The team implemented **dual writes** and then validated results against the master HBase dataset. Dual writing will be a recurring theme of data migrations in this issue:

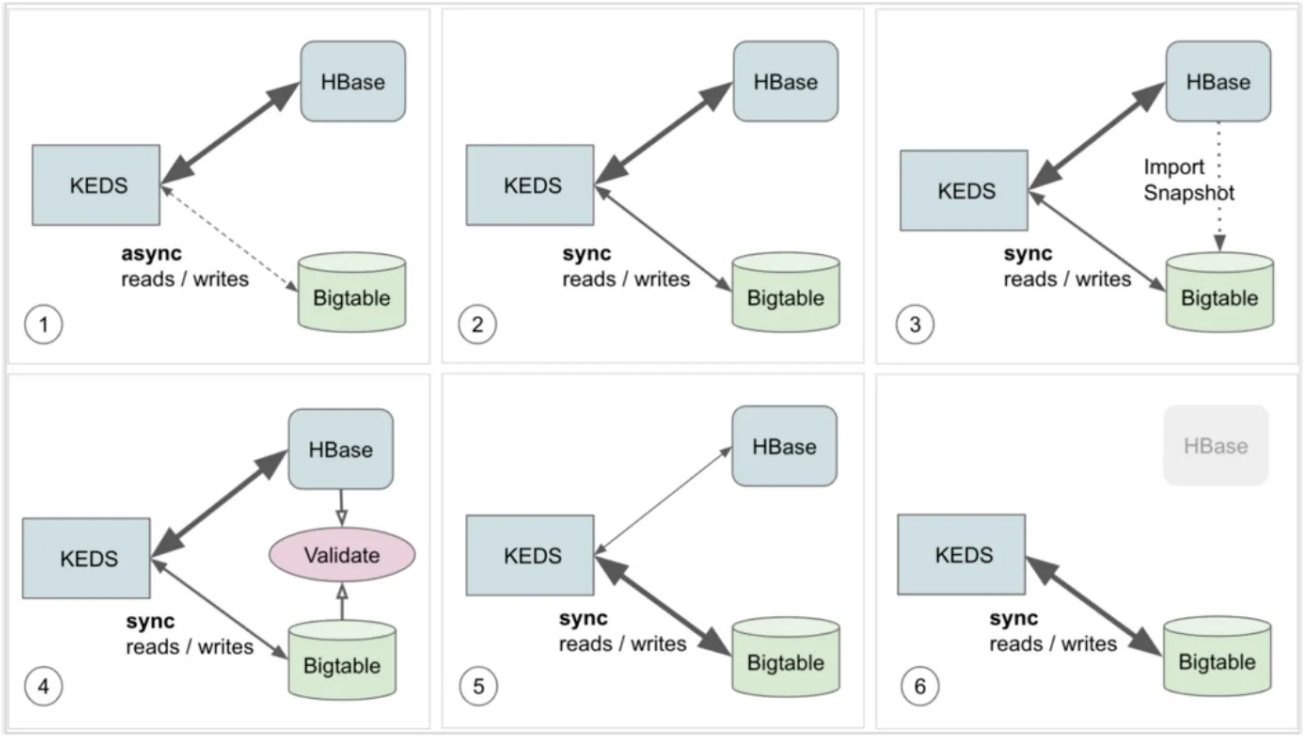


The approach a Box team took to migrate from HBase to Cloud Bigtable: utilizing dual writing, then validating results.

3. Migration plan. Following the performance validation and proof of concept, the team put a 6-stepped, phased plan together:

1. *Async dual reads/writes*, done as a best effort. At this point, nothing happens if Cloud Bigtable goes down.
2. *Synchronous dual reads/writes*. At this point, Cloud Bigtable's performance impacts latency and availability of the service.
3. *Backfill the new service from the old*. Migrate data from HBase to Cloud Bigtable. HBase is still the primary system.
4. *Validate data consistency*. Still with HBase as primary, validate data consistency between the old and the new system.
5. *New system moves to primary*. Cloud Bigtable becomes the source of truth, but sync writes still go to HBase.
6. *Retire the old system*. Once Cloud Bigtable works reliably, retire HBase.

Visualizing these steps:



The 6 steps to migrate from HBase to Google Cloud Bigtable at Box.
Source: Box tech blog.

Read the rest of the article for a lot more details about the migration, including HBase and Cloud Bigtable differences, errors during the migration in Bigtable, CPU spikes, and details of snapshot imports and exports:

I like this approach because it showcases just how many steps it takes to do a large migration, in a responsible way. Not all migrations need so many steps, but in the case of Box, a poor migration could have impacted all their customers. The migration plan is worth considering for any data migration where the stakes are high.

2. Data migration using double writes at Pinterest

Pinterest also decided to move off HBase for their data storage. Ankita Girish Wagh, senior software engineer at Pinterest, summarized her learnings on the way.

The company evaluated more than 10 solutions. Evaluating solutions included benchmarking 3 solutions and sending shadow traffic to them. In the end, Pinterest settled on moving over to **TiDB**. TiDB is an open, unified, distributed SQL database.

The migration from HBase to TiDB took several quarters to complete. This project consisted of the following steps:

1. **Data migration** from HBase to TiDB.
2. **Unified Storage Service implementation**: designing this structured data store, which is powered by TiDB.
3. **API migration**: going from the current Ixai (indexed store) Zen (grapg service) and UMS (key-value store) to Unified Storage Service.
4. **Offline jobs migration:** moving offline jobs from HBase/Hadoop to the TiSpark ecosystem.

The article details the first step, the data migration.

Pinterest first migrated a service with **4TB** of data, which served **14,000 reads/second** and **400 writes/second**. The team chose a zero downtime migration strategy almost identical to the Box team's. The steps, in simplified form:

1. *Import data to the new database*. Set up the initial phase.
2. *Async double writes*. Write the data to both the old and the new database, but do this in an async fashion, not waiting for the write to finish on the new database before returning calls.
3. *Take a snapshot of the old and new database*. This will be used to reconcile data that does not add up.
4. *Reconcile data inconsistencies*. Go through data that had inconsistencies, and resolve these. Most inconsistencies should have been due to the asynchronous writes. If there are cases where the root cause is different, resolve that issue.
5. *Sync double writes*. Data writes are now done synchronously to both the old and the new database.
6. *New database is primary*. TiDB starts to serve reads. Writes are sync to TiDB and async to HBase.
7. *Turn off the old database*. Turn off writes to HBase.

In the article, the Pinterest team breaks out each step using diagrams, outlines more learnings about **data ingestion** challenges, and shares more specific details on the migration steps.

The similarities between the steps the Pinterest team took and the way the Box team approached their migration were revealing. Both migrations were mission-critical, and in both cases, the teams separated asynchronous and synchronous double writing.

3. Navigating the migration chaos at LinkedIn

In 2020, an infrastructure team at LinkedIn decided to deprecate their backend deployment system. This migration impacted **100+ engineers** and **600+ use cases**, making the migration unusually complex.

Lily Wittle, senior software engineer on the Systems Infra team at LinkedIn, shared her learnings on doing large-scale migrations that involve dozens of teams and a large number of engineers.

1. Persuade the clients of the benefits of migration. It's a lot more likely that a team will get additional work done – which is needed for the migration – if they know what's in it for *them*.

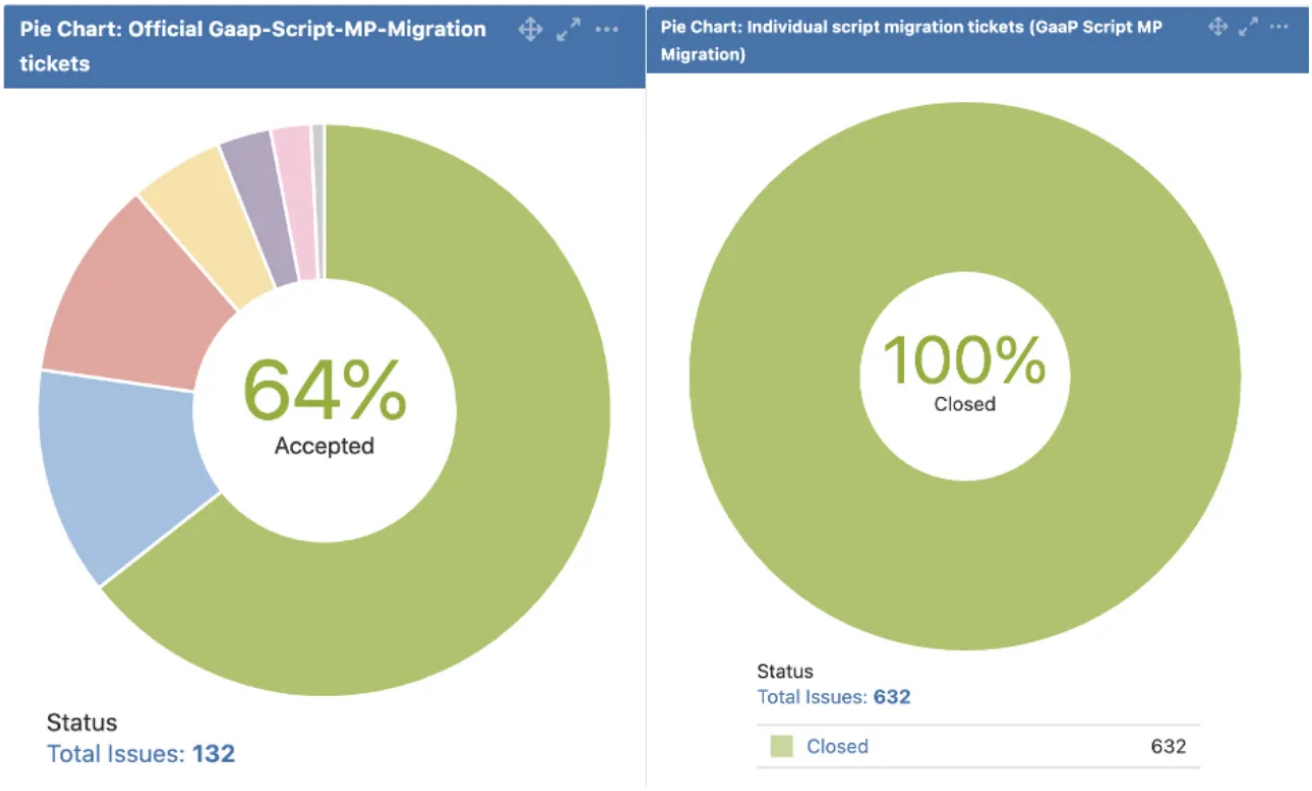
2. Agree on the priority of the migration. Before the migration starts, everyone should agree where the migration sits on the priority list. LinkedIn uses a Horizontal Initiative framework across the company (HI), which helps teams agree on the relative importance of these projects.

3. Write clear instructions. The infrastructure team "dogfooded" their written instructions, ensuring that an engineer with no prior understanding of the infra systems could perform these steps. The team also asked the first few teams doing the migration for feedback, and updated the migration documentation accordingly.

4. Provide automated tooling. The infrastructure team automated the steps that were nearly identical for all teams; for example, setting up directories with a specific structure and copying certain files. They created Python scripts, cutting 1–2 hours of manual work down to a few minutes.

5. Make it easy to get help. The infrastructure team opened a Slack channel at first, then offered office hours for live debugging.

6. Track project progress. The team created a dashboard to track which teams completed the work and which ones were in progress. Setting up such dashboards should be easy, using any project management tool. LinkedIn created these on top of JIRA.



A dashboard visualizing the status of a large migration at LinkedIn.
Source: LinkedIn Engineering.

I liked how this article explained what makes large migrations hard, when 'large' means lots of teams are involved. Projects where many engineering teams are involved are exactly where Technical Program Managers (TPMs) often end up stepping in, in organizations that utilize the TPM role. *Read more about What TPMs do and How to scale engineering organizations with the TPM role.*

4. Executing a migration at scale at Stripe

The previous articles on Box and Pinterest were written in 2021 and 2022, but let's jump back five years, to a 2017 case study from Stripe.

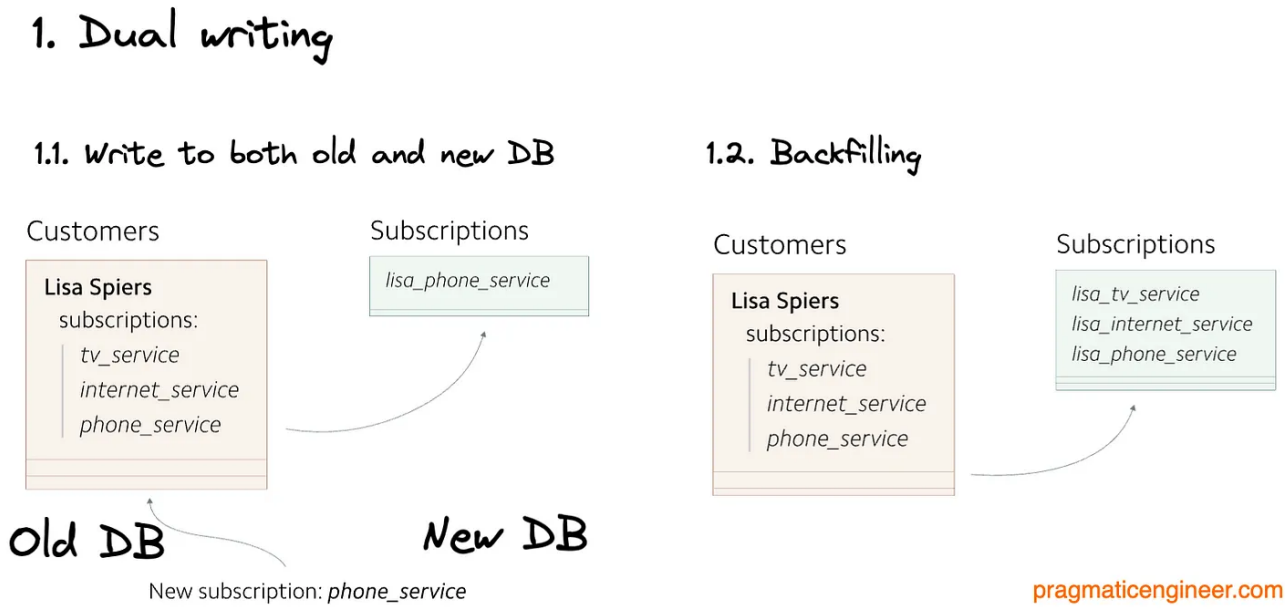
Jacqueline Xu, former product engineer at Stripe, summarized why migrating the Subscriptions database to a different schema was challenging at Stripe, and the steps her team took to complete this project successfully. The biggest challenges were:

- **Scale**: hundreds of millions of subscription objects needed to be migrated.

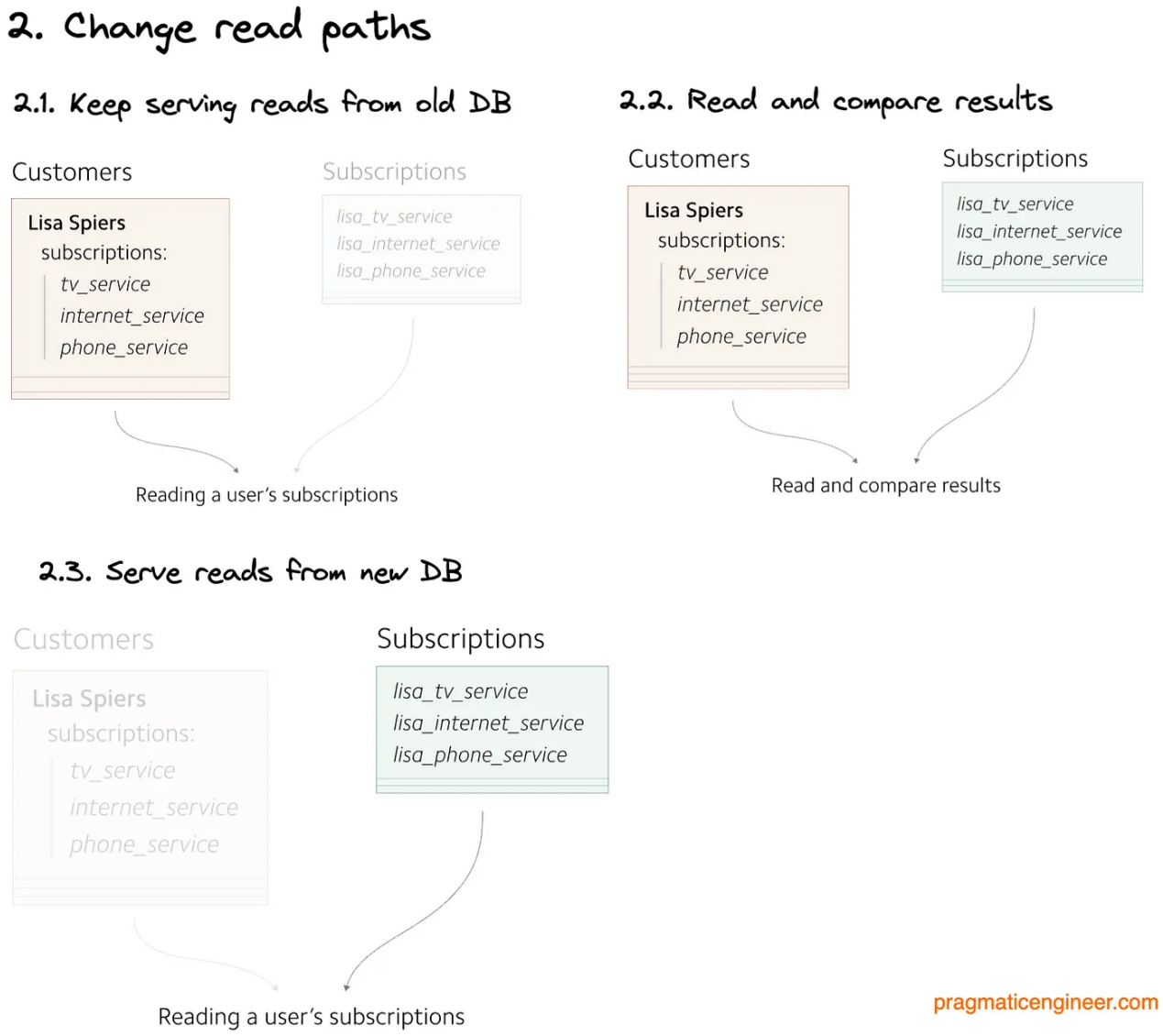
- **Uptime**: during the migration, uptime needed to be as close to 100% as possible.
- **Accuracy**: the Subscriptions table was used in many places across the codebase, and all these usages needed to be updated.

On the surface, this migration was a pretty simple one. All that happened was Subscriptions for each customer were moved out from the Customers table into their own table.

The migration was done in four (what might start to feel predictable) steps:



Step 1: Put dual writing in place. Images source: Stripe Engineering blog.



Step 2: Change read paths so the new database serves reads, after validating that it works correctly. Images source: Stripe Engineering.

Step 3: Change write paths so writes go to the new database first. Step 4: Remove the old data.Migration complete! Images source: Stripe Engineering.

What I enjoyed about this article is how it showcases how a simple migration is not so simple when it's done with a large number of table entries, in an environment where downtime is not an option.

5. Migration lessons from Spotify

Saunak "Jai" Chakrabarti, Senior Director of Engineering at Spotify, and Jenni Lee, User Researcher at the company, shared their candid reflections on migrations. I found myself nodding to many of their observations. Here are a few:

Migrations tend to get stuck. The authors share their observations how migrations start off with good effort, but get "stuck in the mud" over time.

Carrot vs stick approach to get migrations done:

> "A company's engineering culture is going to play a big role in how the [migrations being stuck] problem is perceived.
>
> At Spotify (...) we've seen a lot of success explaining, in creative ways, why a technology upgrade matters and how it helps our engineering community rather than doubling down on a mandate. That's been our style, and a lot of what follows is going to reflect that desire to strike a balance between engineering autonomy and a healthy techscape."

The authors found this strategy efficient:

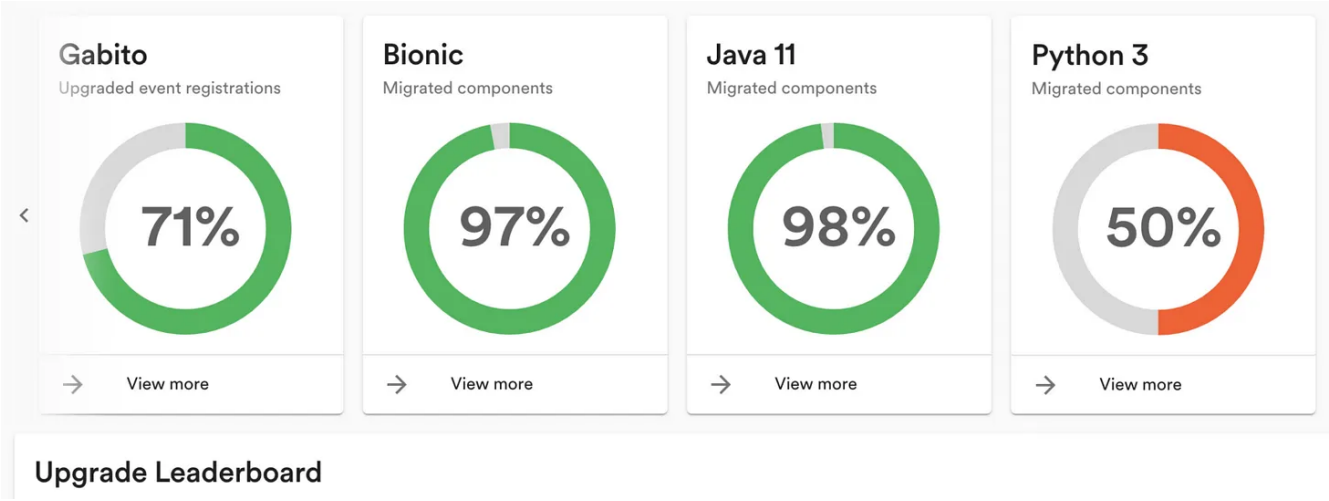
1. Ruthlessly prioritize. Prioritizing is easier said than done when working with highly autonomous teams.

Spotify built a **company-wide migrations map**, with priorities assigned to them. Migrations are communicated in Backstage, Spotify's home-grown, now open-sourced developer portal. Engineers can see which migrations are scheduled for current or future quarters.

2. Each migration to be owned by a product manager (PM). Spotify 'product-ified' migrations, taking a page from the product engineering book. They put these in place for migrations:

- *A product manager is accountable*. Each migration has a PM owner driving this migration.
- *Test before rollout*. Alpha and beta tests are common with migrations, just like they are for products.
- *Training programs*. Where it makes sense, engineers work with Tech Learning to create training resources and programs.

- *Explain the value of the migration*. Each migration communicates the values of this migration, often with the help of product marketing managers (PMMs).
- *Know your customer*. PMs know customers and do things like segment customers.
- *Gamify*. Spotify added a fun, competitive angle to migrations by creating leaderboards for migrations.



Spotify's migration leaderboards. Gamifying migrations can motivate people to 'beat' the other team by finishing their migrations first. Source: Spotify Engineering.

3. Automate, automate, automate! Spotify automates as much as possible. The best migrations are ones where customers don't know much about it until it's ready. A good example is creating automated pull requests to upgrade component versions. Teams just need to inspect and approve these pull requests – or give feedback if they see issues with the change.

Read the full article for more details on each of these steps:

I really, really like a lot of what I heard Spotify do. Even discounting how engineering blogs paint a rosier picture than how things really are, and even if only some Spotify teams follow the steps outlined in the article, those teams are well ahead of how most teams in the industry execute on migrations.

Product managers being involved with migrations is something more teams should explore. A PM brings a much-needed customer-centric approach that is

almost always missing when engineers do the migrations. PMs will think about customer personas, segmentation, gathering feedback, and communicating value.

I'm not saying that PMs should always lead migrations – the work is very engineering heavy, after all – but I am saying that pulling them in to support is almost always a win-win. The PMs learn more about the technical challenges with migrations, and they can help the engineering team become more customer-centric, treating other engineers needing to do migration work as customers, not just as fellow developers. *Read more about advice in the article Working with product managers as an engineering manager or engineer.*

6. Executing a critical migration safely at DoorDash

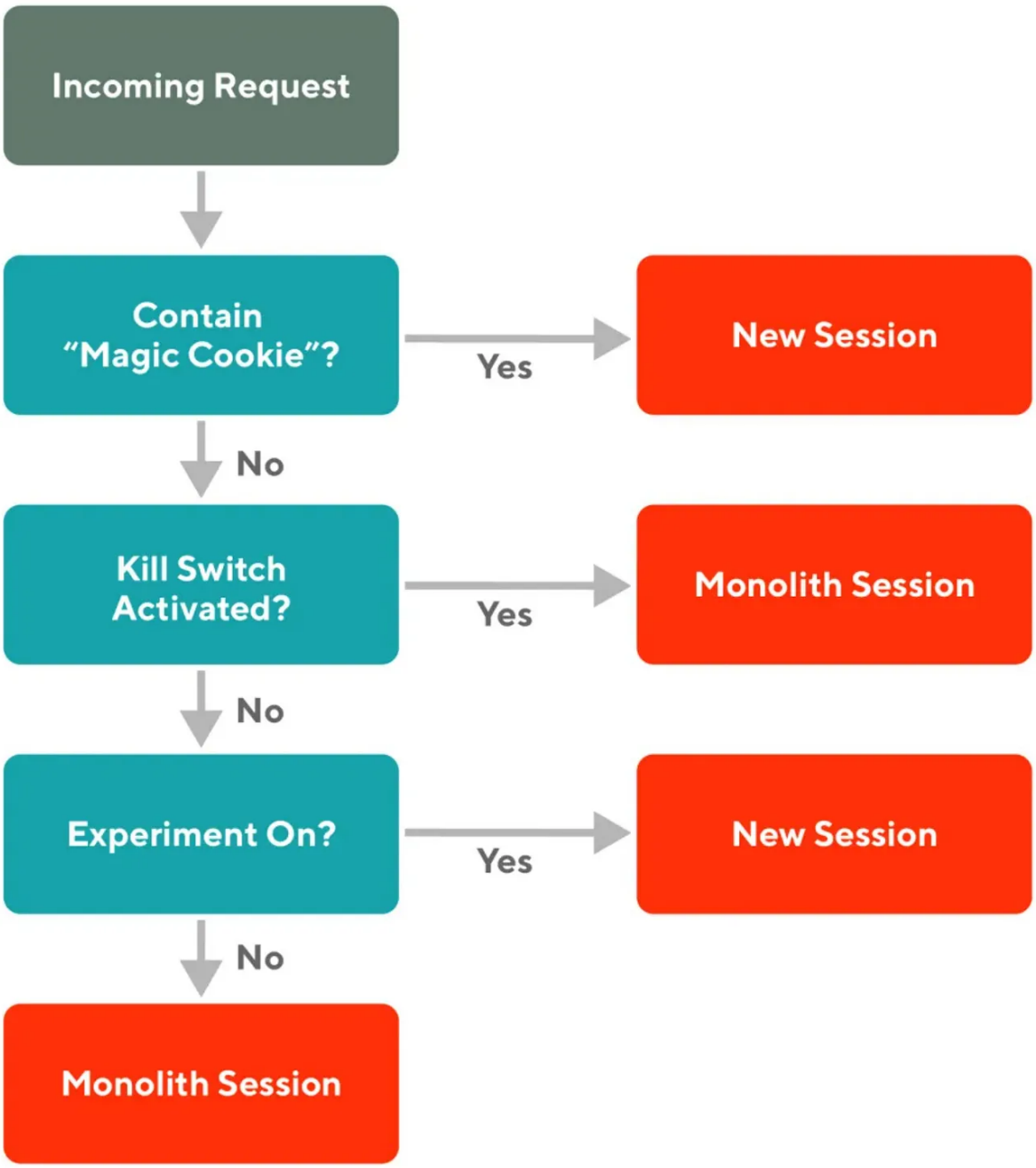
DoorDash software engineers Sin Ko and Li Pei wrote about their learnings as DoorDash re-architected their platform, moving the complex session management system from their monolith, migrating this feature to microservices.

Previously, the concept of sessions was an **implicit dependency** in the monolith. The session functionality included authentication, authorization storing information about the logged in user, and signing out. The team at DoorDash realized that the sessions functionality has lots of key dependencies related to this, like security, or customer experience. Moving sessions to their own service would mean that sessions would become an **explicit dependency**: other modules would utilize sessions through a clear set of API boundaries.

Once the team decided on how to build the new session service, it was time to decide on the migration strategy. Here are goals that the team set:

1. Make the migration safe. Allow for a **rollback** to happen if the migration was paused, or something unexpected came up. This problem was complex because the rollout needed to be synchronized on the client side and on the server side.

The team implemented a **kill switch** as part of the migration. If the kill switch was activated, a full rollback to the pre-migration state would have happened.



To make the migration safe, the team at DoorDash put multiple checks in place on the server side. The "magic cookie" allowed for manual testing, the kill switch allowed for full rollback, and they utilized their experimentation framework for a partial rollout. Source: DoorDash Engineering.

2. Make the migration observable and measurable. The team at DoorDash partnered with Data Analytics to measure conversion rates during the migration. They made sure that if there was a regression in business metrics during the migration, they were notified.

As the team wrote, making the migration measurable reduced the number of decisions to make:

> 'With these data-driven guardrails in place, we were able to take the guesswork out of the rollout plan, helping us to decide when our next ramp up should be, and how much of an increment we could take confidently.'

3. Let other teams know about the migration. To minimize disruption, the team informed the engineering and product teams about the work they were doing.

They also shared their dashboards with other product teams, and asked for help and feedback, in case other product teams saw strange things happening.

Read the full article for more details about this migration, and to learn why the team went back to the drawing board, scrapping their original migration code, so that customers would not be disrupted.

What I liked about this case study is how it described a more complex migration and rollout, one where we were not talking only about a backend migration, but one that had an impact on the clients, as session IDs are also stored on the client side.

Planning for partial or full rollbacks is especially important for client–server migrations. You'll notice that with backend-only data migrations – like the ones Box or Pinterest executed – there was no need for rollback plans, as the rollout moved forward only after the team had validated that everything worked fine.

However, with migrations that involve the client-side and server-side as well, there's a fair chance that engineers will notice the migration going wrong only when this rollout is already midway. Having rollback mechanisms in place is essential in these cases. The kill switch might not be needed for all migrations, but for critical ones, where a corrupted system needs to be restored to health quickly, a killswitch is useful.

Takeaways

In Migrations Done Well, I argued how migrations are an overlooked area in software engineering:

> ‘Migrations are one of the most overlooked topics in software engineering, especially at high-growth startups and companies. As a company’s operations grow, new systems and approaches are adopted to cope with extra load, more use cases, or more constraints. From time to time, engineers need to migrate over from an old system or old approach to a new one.’

In that article, we covered much of the theory behind great migrations. Still, over the past months, I found myself bookmarking case studies which I felt provided practical explanations on how various tech companies tackled migration challenges.

Experienced software engineers consider migrations hard, but also interesting. This is why there are so many case studies describing migrations across a variety of engineering blogs. Engineers should rightfully be proud when finishing a complex migration, and there’s plenty to learn from each of these rollouts.

I hope you will bookmark this issue – together with Migrations done well – and come back to it the next time you’ll need to migrate to a new system. Once you successfully roll out the new service and retire the old one, I’d suggest you pause and reflect. What went well with that migration? What could have been better? What did you learn?

As before, I’m still experimenting on the format of Real-World Engineering Challenges. This is the first issue where I collected engineering challenges that are around the same theme. I’m interested in what you think. Please leave feedback here, and you can add a comment after choosing how you liked this issue.

How would you rate this issue? 🤔

Amazing • Great • Good • OK • So-so

117 Likes

Comments

Write a comment...